I. Convolutional Neural Networks (CNNs)

1. **Convolution Operation:**

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

- I: Input image
- K: Kernel/filter
- (i, j): Coordinates of the output feature map

2. **Output Size for Convolution:**

$$O_{\text{width}} = \frac{W - F + 2P}{S} + 1$$

- W: Input width
- F: Filter size
- P: Padding
- S: Stride

3. **Receptive Field**:

$$R = 1 + (F - 1)S$$

- F: Filter size
- S: Stride

4. **Max Pooling** (for pooling layers):

$$O(i, j) = \max_{m,n}\{I(i + m, j + n)\}$$

- Applies the max operation over regions of the input.

5. **Global Average Pooling**:

$$O = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} I(i, j)$$

- H, W: Height and width of the input.

6. **Batch Normalization**:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

- μ: Mean, σ: Variance, γ, β: Learnable parameters.

7. **Dropout Regularization:**

$$\text{Dropout rate} = \frac{1}{p}$$

- p: Probability of keeping a unit.

II. Activation Functions

8. **ReLU (Rectified Linear Unit):**

$$f(x) = \max(0, x)$$

9. **Leaky ReLU:**

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

- α: Small constant (e.g., 0.01)

10. **Sigmoid:**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

11. **Tanh:**

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

12. **Softmax Function:**

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

- Converts logits to probabilities for classification.

III. Loss Functions

13. **Cross-Entropy Loss:**

$$\mathcal{L}_{\text{CE}} = -\sum_{i=1}^{C} y_i \log(p_i)$$

- y_i: True label, p_i: Predicted probability.

14. **Mean Squared Error (MSE):**

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

15. **Binary Cross-Entropy:**

$$\mathcal{L}_{\text{BCE}} = -y \log(p) - (1 - y) \log(1 - p)$$

16. **Hinge Loss (for SVMs):**

$$\mathcal{L}_{\text{hinge}} = \max(0, 1 - y_i \cdot f(x_i))$$

- y_i: True label (+1 or -1), $f(x_i)$: Predicted score.

IV. Optimization and Regularization

17. **Gradient Descent**:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}$$

- η: Learning rate.

18. **Adam Optimizer** (first moment and second moment estimates):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} m_t$$

19. **L2 Regularization** (Weight Decay):

$$\mathcal{L}_{\text{regularized}} = \mathcal{L} + \lambda \sum_{i=1}^{N} w_i^2$$

- λ: Regularization strength.

20. **Learning Rate Decay**:

$$\eta_t = \eta_0 \cdot \frac{1}{1 + \alpha t}$$

21. **Momentum**:

$$v_t = \mu v_{t-1} + \eta \nabla_\theta \mathcal{L}$$

- μ: Momentum term.

V. Image Preprocessing and Augmentation

22. **Image Normalization:**

$$I' = \frac{I - \mu}{\sigma}$$

- Standardizes image pixels.

23. **Translation Augmentation:**

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

- t_x, t_y: Translation in x and y directions.

24. **Rotation Augmentation:**

$$R = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- θ: Angle of rotation.

25. **Scaling Augmentation:**

$$S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- s_x, s_y: Scaling factors.

VI. Variational Autoencoders (VAE)

26. **Reparameterization Trick:**

$$z = \mu + \sigma \cdot \epsilon$$

27. **KL Divergence in VAE:**

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2} \sum_{i=1}^{D} \left(1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2\right)$$

28. **VAE Loss:**

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{reconstruction}} + \mathcal{L}_{\text{KL}}$$

VII. Object Detection and Segmentation

29. **Intersection over Union (IoU):**

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

30. **Non-Maximum Suppression (NMS):**

- Removes redundant bounding boxes based on IoU threshold.

31. **Dice Coefficient (Segmentation):**

$$\text{Dice} = \frac{2 \cdot |A \cap B|}{|A| + |B|}$$

VIII. Image Filtering

32. **Gaussian Filter:**

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

- Used for image smoothing.

33. **Sobel Edge Detection (Gradient Filter):**

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

34. **Laplacian of Gaussian:**

$$\nabla^2 G(x, y) = \frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2}$$

IX. Image Transforms

35. **Fourier Transform:**

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

36. **Inverse Fourier Transform:**

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

1. Reparameterization Trick (VAE Latent Space Sampling):

The latent variable z in the VAE is computed using the formula:

$$z = \mu + \sigma \cdot \epsilon$$

Where:

- μ is the mean vector produced by the encoder.
- σ is the standard deviation vector produced by the encoder.
- ϵ is a random noise vector sampled from the standard normal distribution $N(0, I)$ where I is the identity matrix.
- $\cdot$ denotes element-wise multiplication.

This formula is key to problems involving latent space in a VAE setup, like the one you encountered. In your case:

$$z_i = \mu_i + \sigma_i \cdot \epsilon_i \quad \text{for each element } i \text{ of the vectors.}$$

2. Cross-Entropy Loss for Classification:

For image classification problems, the cross-entropy loss is commonly used, defined as:

$$\mathcal{L}_{\text{cross-entropy}} = -\sum_{i=1}^{C} y_i \log(p_i)$$

Where:

- C is the number of classes.
- y_i is a binary indicator (0 or 1) if the class label i is the correct classification for the input.
- p_i is the predicted probability for class i (from the softmax output).

This formula is essential for multi-class classification problems, especially when computing probabilities in neural networks.

3. KL Divergence (VAE Loss Term):

In a VAE, the loss function consists of two terms: the reconstruction loss and the **KL divergence**. The KL divergence measures how much the learned latent distribution differs from a prior Gaussian distribution. The formula is:

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2} \sum_{i=1}^{D} \left(1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2\right)$$

Where:

- D is the dimensionality of the latent space.
- μ is the mean vector from the encoder.
- σ is the standard deviation vector from the encoder.

This term helps regularize the learned distribution, ensuring it stays close to a normal distribution.

4. Softmax Function for Probability Distribution:

For multi-class classification, the **softmax function** is used to convert logits into probabilities:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

Where:

- z_i are the logits (raw model outputs).
- C is the total number of classes.
- p_i is the probability of class i.

This formula is used to compute class probabilities in classification tasks and is especially important for answering MCQs based on class predictions.

5. Gradient Descent for Model Optimization:

Gradient descent is the optimization technique used to update weights during training. The weight update rule is given by:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}$$

Where:

- θ_t are the model parameters at step t.
- η is the learning rate.
- $\nabla_\theta \mathcal{L}$ is the gradient of the loss function $\mathcal{L}$ with respect to the model parameters.

This is used to minimize the loss function iteratively.

6. L2 Regularization (Weight Decay):

To prevent overfitting, L2 regularization adds a penalty to the loss function based on the magnitude of the weights:

$$\mathcal{L}_{\text{regularized}} = \mathcal{L} + \lambda \sum_{i=1}^{N} w_i^2$$

Where:

- λ is the regularization parameter.
- w_i are the weights of the model.
- $\mathcal{L}$ is the original loss function.

This regularization encourages smaller weights, making the model less prone to overfitting.

Application to Computer Vision Problems:

1. **For Variational Autoencoders (VAE) Problems:**
   - Use the **reparameterization trick**: $z = \mu + \sigma \cdot \epsilon$, as shown in the example.
   - If asked to compute latent variables or sampling-based calculations, always compute z using this formula.
2. **For Classification or Multi-Class Questions:**
   - Use the **cross-entropy loss** for calculating loss and the **softmax function** to convert logits to probabilities for multiple classes.
3. **For Regularization Questions:**
   - Use **KL Divergence** when working with VAEs and **L2 regularization** to prevent overfitting in classification models.
4. **For MCQs on Optimization:**
   - Understand **gradient descent** and its variants (e.g., SGD, Adam) for answering questions related to training convergence or optimization steps.